

Exploring and Testing APIs with Postman

Report / Documentation

Student Name: Elvire AKAYEZU

Date: 05th/Sep/2026

API Chosen: GitHub API (Gists resource)

Base URL: <https://api.github.com>

Module: Web Infrastructure (Formative_1)

1. API Choice and Base URL

I chose the GitHub API because it provides a genuine, well-documented REST interface with real authentication and full CRUD support on the Gists resource. Another reason is that GitHub's endpoints don't reset their data or impose tight rate limits, which made it reliable to test repeatedly during development. I used the Gists resource specifically because creating, updating, or deleting a gist has no effect on any of my real repositories, making it safe to experiment with.

Operation	Method	URL	Auth Required	Headers	Body Format
Create Gist	POST	{{base_url}}/gists	Yes	Authorization: Bearer {{auth_token}}	JSON: description, public, files
Get Gist	GET	{{base_url}}/gists/{{gist_id}}	Yes	Authorization: Bearer {{auth_token}}	None
Update Gist	PATCH	{{base_url}}/gists/{{gist_id}}	Yes	Authorization: Bearer {{auth_token}}	JSON: description, files
Delete Gist	DELETE	{{base_url}}/gists/{{gist_id}}	Yes	Authorization: Bearer {{auth_token}}	None

2. Authentication Method Used

I authenticated using a GitHub Personal Access Token (PAT), which is GitHub's equivalent of an API key. I generated the token through GitHub's website under Settings → Developer settings → Personal access

tokens, scoping it specifically to the “gist” permission so it could only affect gists, not my other repositories.

Rather than sending a login request through Postman, the token is created manually beforehand and then stored as an environment variable called *auth_token*. I applied it once at the collection level using Bearer Token authentication, with the token field set to github_token *{{auth_token}}*. Every individual request inside the collection is set to “**Inherit auth from parent**,” meaning Postman automatically attaches the Authorization header to each request without me having to configure it separately. This keeps the setup clean and avoids repeating the token across every request.

+

COLLECTIONS

API Assignment

GET READ rqst(GET)

POST Create rqst(POST)

PATCH Update rqst(PAT...

DEL DELETE gist_rqst

PUT Star Gist(PUT))

My Collection

New Collection

ENVIRONMENTS

API_Testing_env

API Assignment

Overview

Authorization

Scripts

Variables

Runs

Publish docs

Share

0

0

This authorization method will be used for every request in this collection. You can override this by specifying one in the request.

Auth Type

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token

{{auth_token}}

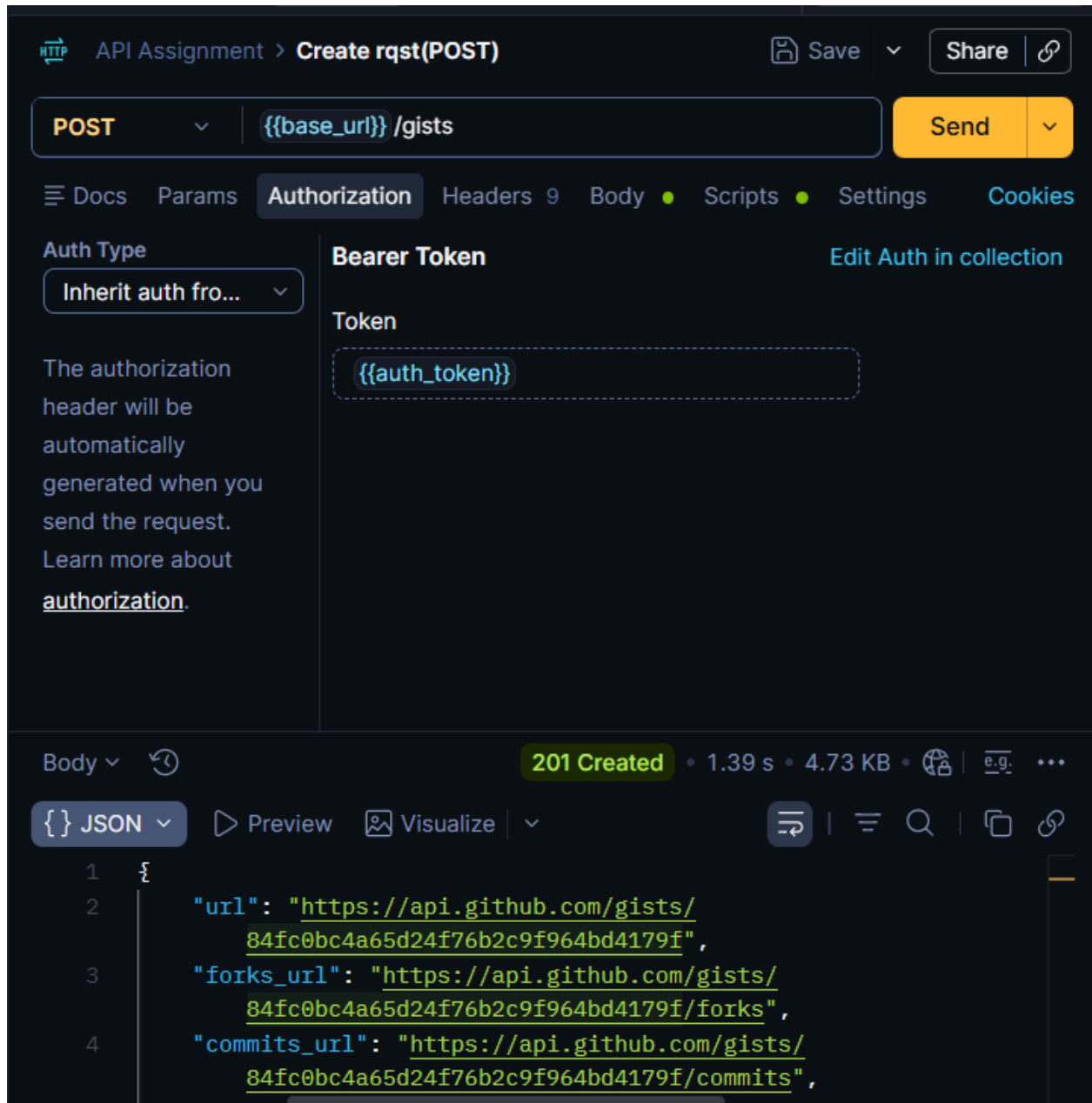


Image showing how the requests, use the authorization by inheriting from the parent(collection), instead of manually making each request have its own authorisation.

3. CRUD Operations

3.1 CREATE (POST)

This request sends a POST to /gists with a JSON body containing a description, a visibility flag, and file content. GitHub responds with a 201 status code and a JSON object containing the new gist's unique ID, which I use in all subsequent requests.

The screenshot displays the Postman interface for an API request. The top bar shows the request is a **POST** to the URL `{{base_url}}/gists`. The **Body** tab is selected, showing a JSON payload:

```
{ "description": "My test gist", "public": true, "files": { "test.txt": { "content": "Hello from Postman" } } }
```

. The **Test Results** section at the bottom indicates a successful response with a status of **201 Created**. Below this, two test assertions are shown as **PASSED**: "Status is 201" and "Response has id".

API Assignment > **Create rqst(POST)** Save Share

POST `{{base_url}}/gists` Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw **JSON** Schema Beautify

```
1 { "description": "My test gist", "public": true, "files": { "test.txt": { "content": "Hello from Postman" } } }
```

Test Results 201 Created • 1.35 s • 4.73 KB • 🌐 🔗 ⋮

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#) Run Collection ×

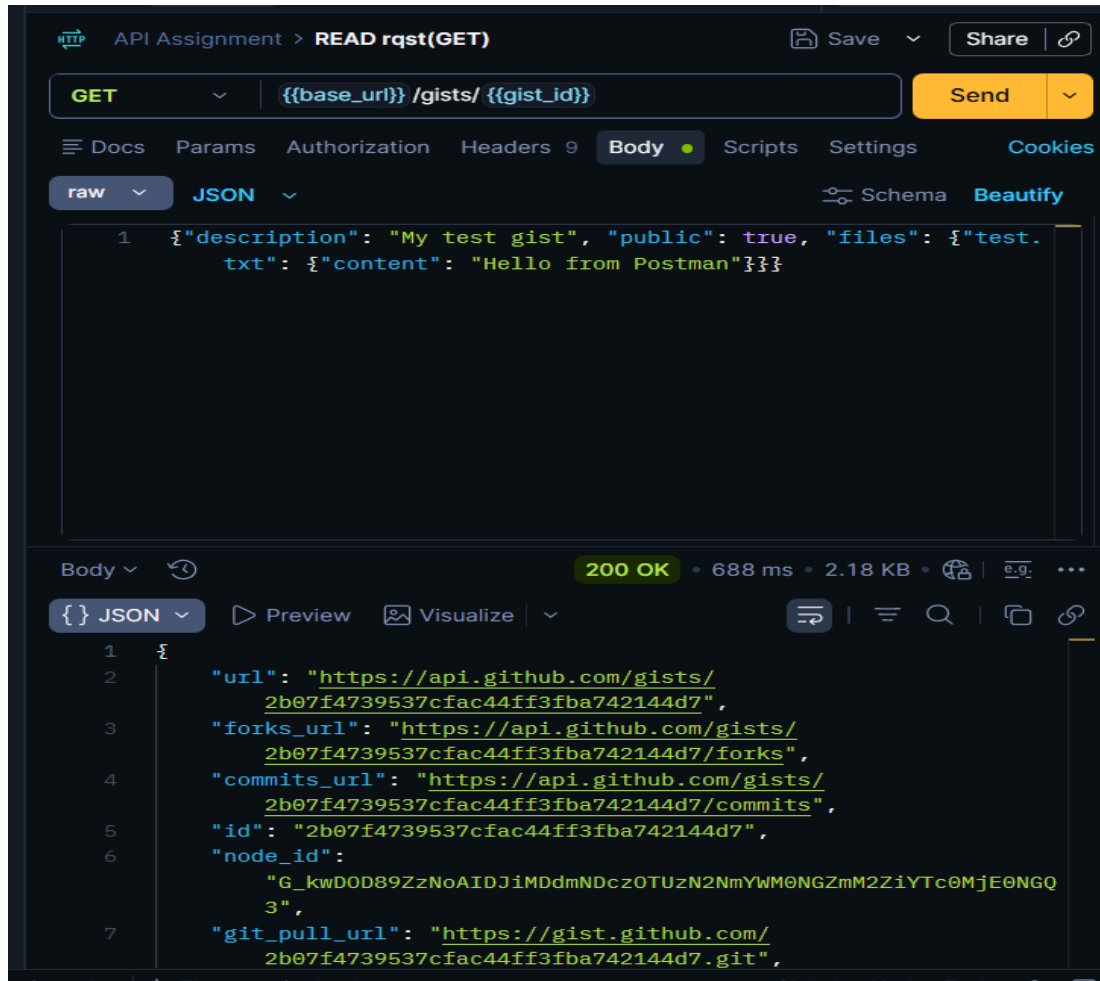
Filter Results 🔄

PASSED Status is 201

PASSED Response has id

3.2 READ (GET)

This request sends a GET to `/gists/{{gist_id}}`, using the ID captured from the Create step. GitHub responds with a 200 status and the full gist object, confirming the data was created correctly and is retrievable.



3.3 UPDATE (PATCH)

This request sends a PATCH to `/gists/{{gist_id}}` with an updated description and file content. The response returns a 200 status and reflects the new values, confirming the update was applied.

HTTP

API Assignment > Update rqst(PATCH)

Save

Share

PATCH

{{base_url}} /gists/ {{gist_id}}

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {"description": "Updated description", "files": {"test.txt":
    {"content": "Updated content"}}}
```

Body

200 OK • 990 ms • 2.33 KB • e.g. ...

{{}} JSON

Preview

Visualize

```
1 {
2   "url": "https://api.github.com/gists/
3     84fc0bc4a65d24f76b2c9f964bd4179f",
4   "forks_url": "https://api.github.com/gists/
5     84fc0bc4a65d24f76b2c9f964bd4179f/forks",
6   "commits_url": "https://api.github.com/gists/
7     84fc0bc4a65d24f76b2c9f964bd4179f/commits",
8   "id": "84fc0bc4a65d24f76b2c9f964bd4179f",
9   "node_id":
10     "G_kwDOD89ZzNoAIDg0ZmMwYmM0YTY1ZDI0Zjc2YjJjOWY5NjRiZDQxNz1
11     m",
12   "git_pull_url": "https://gist.github.com/
13     84fc0bc4a65d24f76b2c9f964bd4179f.git",
```

Console

All systems are go!

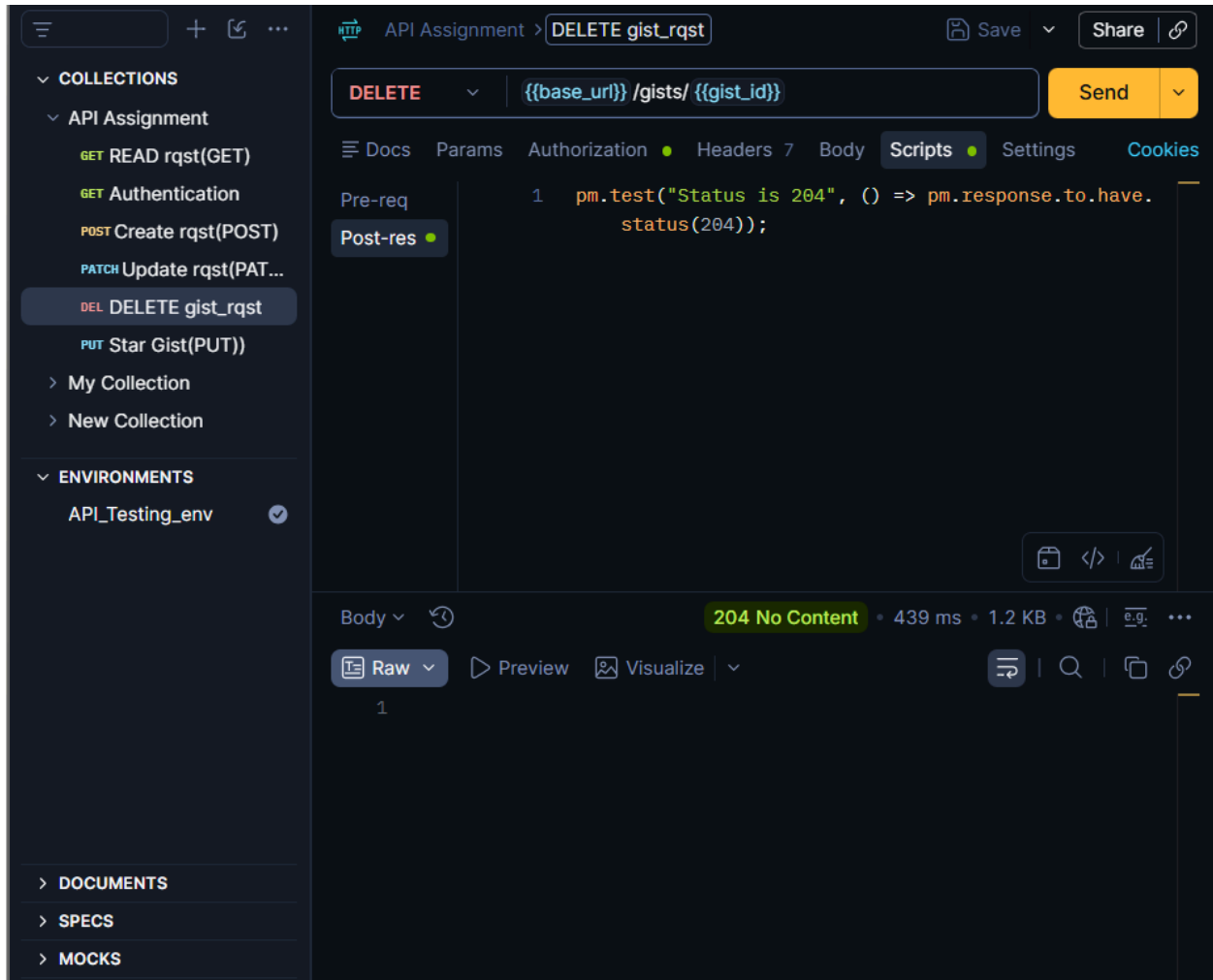
Globals

Vault

Tools

3.4 DELETE

This request sends a DELETE to `/gists/{{gist_id}}`. GitHub responds with a 204 No Content status, which is the expected success response for a deletion. The absence of a response body simply means there's nothing left to return.



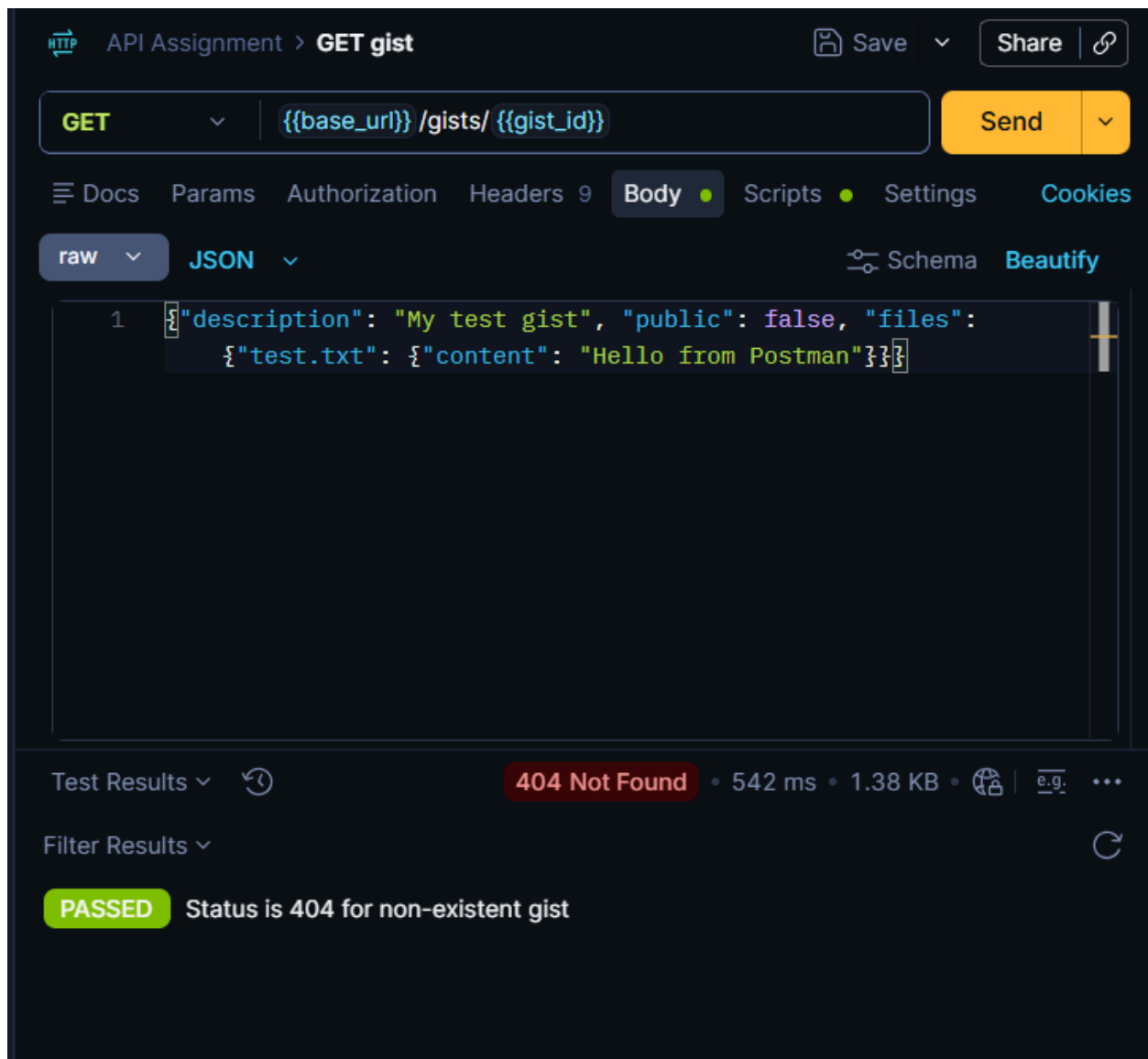
3.5 Error Handling Tests

In addition to demonstrating successful CRUD operations, I built tests that check expected failure scenarios, since verifying an API's error behaviour is as important as verifying its success behaviour.

Invalid Gist ID: I sent a GET request to a made-up, non-existent gist ID (e.g. `93ea9cfeed0dc10b338724eb90e0057`), or I use the previous ID that was deleted by DEL request; it returns the

same error, as it is also a non-existent ID. Since no gist exists with that ID, GitHub correctly returned a 404 Not Found status.

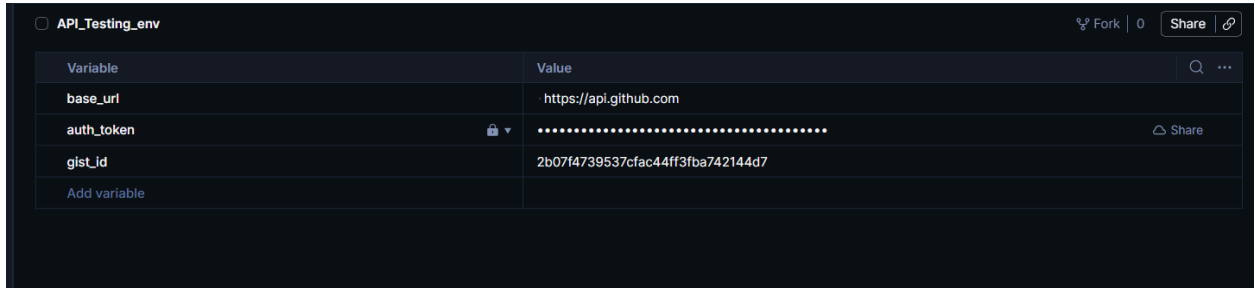
```
pm.test("Status is 404 for non-existent gist", () => pm.response.to.have.status(404));
```



4. Postman Environment & Variables

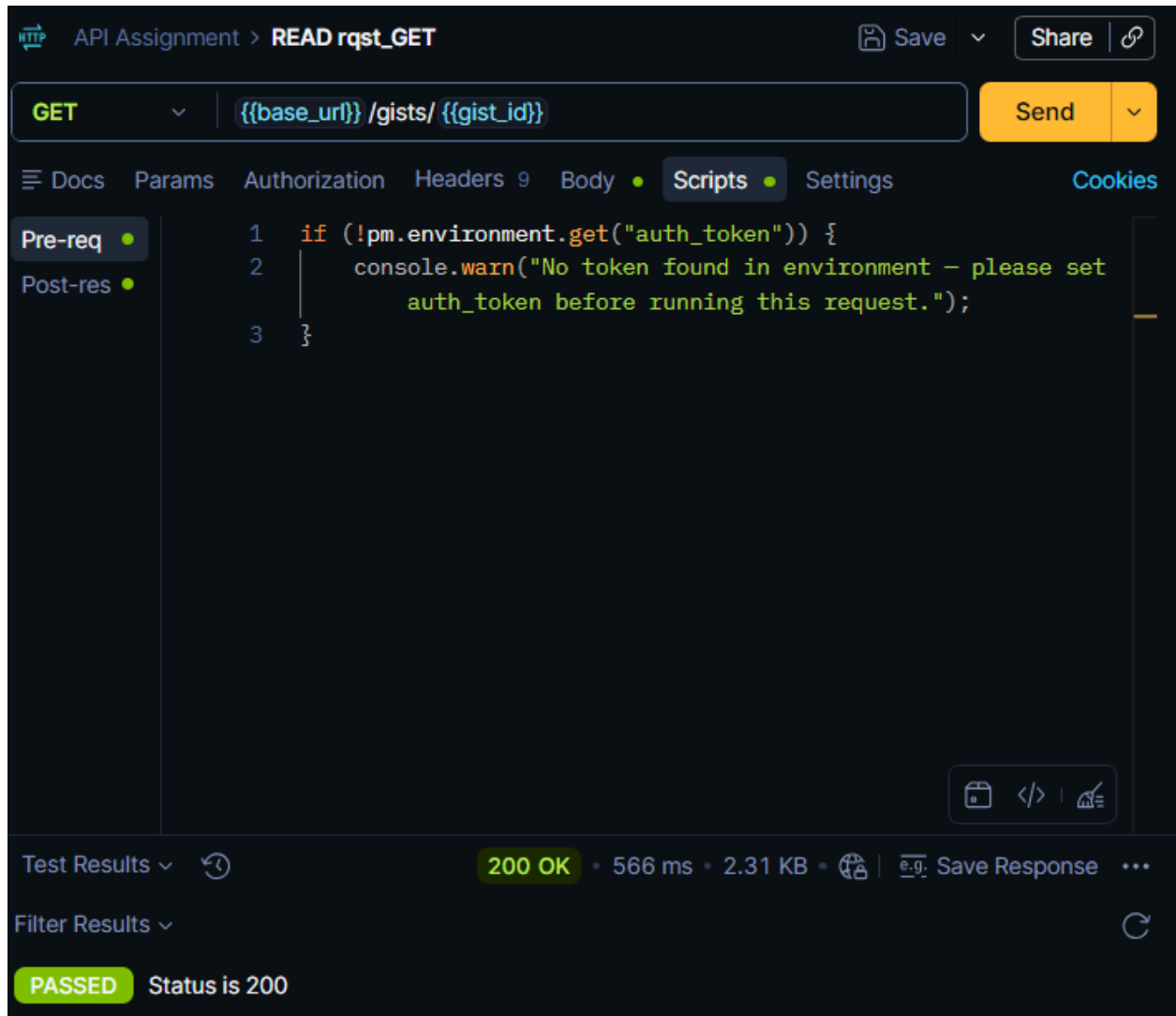
I created three environment variables: *base_url* (the API's root URL), *auth_token* (my authentication token), and *gist_id* (the ID of the gist created during testing). Using variables instead of hardcoding these values, lets me update my token or switch environments without editing every request individually, and it keeps sensitive values like the token out of the request bodies.

The `gist_id` variable is set automatically rather than manually: a test script attached to the Create Gist request extracts the `id` field from the response and stores it directly into the environment using `pm.environment.set()`. This means the Get, Update, and Delete requests always reference the most recently created gist without me copy-pasting IDs by hand.

A screenshot of a GitHub Actions environment configuration interface. At the top, it says "API_Testing_env" with a search icon, a "Fork" button with "0" forks, and a "Share" button. Below this is a table with two columns: "Variable" and "Value". The table contains three rows: "base_url" with value "https://api.github.com", "auth_token" with a masked value "....." and a "Share" button, and "gist_id" with value "2b07f4739537cfac44ff3fba742144d7". At the bottom of the table is an "Add variable" link.

Variable	Value
base_url	https://api.github.com
auth_token	 Share
gist_id	2b07f4739537cfac44ff3fba742144d7
Add variable	

Another thing is that. "Since GitHub's Personal Access Token is generated manually on GitHub's website rather than through a login endpoint, true automatic token refresh via a *Pre-request Script* *doesn't apply to this API*. To still demonstrate the underlying concept, I added a **Pre-request Script** that checks whether the token variable is present before a request runs:



"This shows the same principle, which is validating token availability before the request executes.

5. Automation & Collection Runner

I wrote test scripts on each request to automatically verify success rather than checking responses manually. The Create request checks that the status code is 201 and that the response contains an id field, then saves that ID for reuse. The Get, Update, and Delete requests include similar status-code checks (200, 200, and 204 respectively).

To test the full workflow at once, I used Postman's Collection Runner, running the requests in the order Create → Get → Update → Delete. This order matters because each later request depends on the gist_id set by the Create request; running them out of order would cause the Get, Update, or Delete requests to fail since no ID would exist yet. The Runner executed all four requests successfully with all tests passing.

COLLECTIONS

API Assignment

POST Create rqst(POST)

PUT Star Gist(PUT))

GET READ rqst(GET)

PATCH Update rqst(PAT...

DEL DELETE gist_rqst

My Collection

New Collection

ENVIRONMENTS

API_Testing_env

DOCUMENTS

SPECS

MOCKS

DATASETS

Run orderMock

Run SequenceDeselect AllSelect AllReset

1

POST

Create rqst(POST)

2

PUT

Star Gist(PUT))

3

GET

READ rqst(GET)

4

PATCH

Update rqst(PATCH)

5

DEL

DELETE gist_rqst

Run type

Functional

Validate request correctness and test

Performance

Measure response times and load be

Run method

Local

Iteration data

None

Iterations

1

Settings

Persist responses for a session

Turn off logs during run

Stop run if an error occurs

Keep variable values

Run collection without using store

Save cookies after collection run

Add a delay of0ms

between requests

Start run

API Assignment - Run results

Ran today at 08:56:01 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	API_Testing_env	1	4s 759ms	7	0	791 ms

All 7 Passed 7 Failed 0 Skipped 0 Errors 0 Console log

Iteration 1

- POST Create rqst(POST)**
<https://api.github.com/gists>
 281 • 1383 ms • 4.846 KB 2
 PASS Status is 201
 PASS Response has id
- GET READ rqst(GET)**
<https://api.github.com/gists/d7af92d5d6e23dafa3ab2752eee6aed0>
 280 • 417 ms • 2.365 KB 2
 PASS Status is 200
 PASS Response has id
- PATCH Update rqst(PATCH)**
<https://api.github.com/gists/d7af92d5d6e23dafa3ab2752eee6aed0>
 280 • 981 ms • 2.388 KB 2
 PASS Status is 200
 PASS Description was updated
- DELETE DELETE gist_rqst**
<https://api.github.com/gists/d7af92d5d6e23dafa3ab2752eee6aed0>
 284 • 383 ms • 1.227 KB 1
 PASS Status is 204

6. Reflection / Conclusion

Through this assignment, I learned how token-based authentication works in practice, and specifically how Postman's “inherit auth from parent” setting lets a single token apply across an entire collection without repetition. The main challenge I faced was understanding why my individual requests didn't need explicit authorization. Once I checked each request's Authorization tab and saw it was inheriting from the collection, this made sense.